

第7章 文件I/O

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立 C 的 stdio 到 Go os/bufio 包的 API 对照表,不再为"哪个函数对应哪个"发愁
- 掌握文本读写:ReadFile/WriteFile/OpenFile、追加模式、错误处理(含 os.IsNotExist)
- 掌握大文件按行流式读取(bufio.Scanner),理解内存占用对比
- 掌握二进制读写与 encoding/binary,理解"C 直接 fwrite 结构体的危险"
- 掌握路径与目录:filepath.Join、os.ReadDir、MkdirAll、跨平台分隔符
- 掌握 JSON 序列化:Marshal/Unmarshal/MarshalIndent、结构体标签、流式 Decoder
- 掌握标准流与命令行参数:os.Stdin/Stdout/Stderr、os.Args、flag 包

💡 从 C 到 Go:本章主题如何迁移

C 的文件 API 是 stdio: `fopen` (模式字符串 `"r"/"w"/"a"`)、`fread/fwrite` (每次指定元素大小与个数)、`fgets` (行读取)、`fclose`。它有两个痛点:①模式字符串是约定,拼错只是行为怪异;②`fread` 返回"读了多少元素",你必须自行处理"部分读"与错误区分;③文件不存在时 `fopen` 返回 NULL,错误原因存在 `errno` 里,查起来费劲。

Go 的 os 包:打开文件返回 `(*os.File, error)` ——错误直接是返回值, `os.IsNotExist(err)` 精准判断"文件不存在";读文件 `Read` 返回 `(n, err)`,读到末尾返回 `io.EOF`,约定清晰;更常用的是一行搞定的小函数: `os.ReadFile` 整个读入、`os.WriteFile` 整个写出——C 里"读整个文件"要自己量长度、分配、循环读,Go 一个函数。

三条主线对照:C 的 `fopen/fread/fwrite/fclose` → Go 的 `os.Open/Read/Write/Close` (配合 `defer`);C 的 `fgets` → Go 的 `bufio.Scanner`;C 的 `fwrite(&struct)` → Go 的 `encoding/binary` + 显式字段编码(C 直接写结构体是 UB 级危险:对齐、字节序、字段布局全凭平台)。序列化方面,C 没有标准 JSON,要么手写解析器要么引第三方;Go 标准库 `encoding/json` 直接可用。

一句话主线: C 的 I/O 是"函数 + 模式约定 + `errno`",Go 的 I/O 是"函数 + 类型 + `error` 返回值"——同样的能力,错误从"查全局"变成"返回值里带着"。

📦 前置知识自查

- C 的 `fopen/fread/fwrite/fgets/fprintf/fclose` 与文件模式
- `errno`、`perror` 的错误处理方式;EOF 的判定
- 结构体对齐、字节序(endian)的基本概念
- 第 4 章的 `defer`(文件关闭的标准姿势)、第 5 章的 `buffer` 概念

🚀 本章学习路线

1. **第一阶段:**§7.1–§7.3,掌握文本读写三件套:ReadFile、Scanner 流式读、OpenFile 追加
2. **第二阶段:**§7.4–§7.6,二进制、目录、JSON 三个独立主题,各跑一个示例
3. **第三阶段(实验):**§7.7–§7.8 与章末实验:日志解析统计工具,把整章 API 织进一个真实工具
4. **验收标准:**能不看资料写出"读一个配置文件并容错解析"的完整代码

本章知识导图

- §7.1 API 对照表:C 的 stdio → Go 的 os/bufio
- §7.2 文本读写:ReadFile/WriteFile/OpenFile/追加
- §7.3 大文件流式读取:bufio.Scanner 与内存对比
- §7.4 二进制 I/O:encoding/binary 与"C 直接 fwrite 结构体"的危险
- §7.5 路径与目录:filepath、os.ReadDir、MkdirAll
- §7.6 JSON 序列化:Marshal/Unmarshal/标签/流式 Decoder
- §7.7 标准流与命令行参数:os.Args 与 flag 包
- §7.8 CSV 与日志:生产场景实战

§7.1 API 对照表:从 stdio 到 os 包

C 的写法	Go 的写法
FILE* f = fopen(path, "r"); NULL 表示失败	f, err := os.Open(path); err != nil 表示失败
fopen(path, "w") / "a"	os.Create(path) / os.OpenFile(path, os.O_APPEND os.O_WRONLY, 0644)
fread(buf, size, n, f),返回值 是"读到的个数"	n, err := f.Read(buf);读到末尾 err == io.EOF
fgets(line, n, f)	scanner := bufio.NewScanner(f); scanner.Scan()
fprintf(f, fmt, ...)	fmt.Fprintf(f, fmt, ...) (第一个参数换成 io.Writer)
fclose(f) 每处手动	defer f.Close() 一行保证
errno + perror()	err 返回值 + os.IsNotExist(err) 等分类函数
fwrite(&s, sizeof(s), 1, f) 直接 写结构体	encoding/binary 显式编码每个字段(对齐/字节序受控)

★ 重点:读文件的三个层次

①小文件: os.ReadFile 一次读入(内部帮你打开+读完+关闭);②大文件/流式: os.Open + bufio.Scanner 按行处理;③需要精细控制: os.OpenFile + Read / Write 循环 + 显式 Close。生产上 90% 场景落在前两层,别一上来就写 C 式手循环。

§7.2 文本读写:三件套

示例 7-1 读写文本文件

```
package main

import (
    "fmt"
    "os"
)

func main() {
    // 写文件:整个写出(不存在则创建,存在则截断)
    err := os.WriteFile("hello.txt", []byte("你好,Go 文件!\n"), 0644)
    if err != nil {
        fmt.Println("写入失败:", err)
        return
    }

    // 读文件:整个读入
    data, err := os.ReadFile("hello.txt")
    if err != nil {
        fmt.Println("读取失败:", err)
        return
    }
    fmt.Print(string(data))

    // 追加模式:OpenFile 显式指定标志
    f, err := os.OpenFile("hello.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
    if err != nil {
        fmt.Println("打开失败:", err)
        return
    }
    defer f.Close() // 无论后续如何,句柄必释放
    if _, err := f.WriteString("追加一行\n"); err != nil {
        fmt.Println("追加失败:", err)
    }
}
```

⚠ 易错点 7-1:错误判断用 `os.IsNotExist`,不要比字符串

不要写 `err.Error() == "file does not exist"` —— 错误消息随平台与 Go 版本变化。正确姿势:`os.IsNotExist(err)` (以及 `os.IsPermission`、`os.IsTimeout`)。Go 1.13+ 推荐用 `errors.Is(err, os.ErrNotExist)`,兼容包装错误(第 9 章)。

§7.3 大文件流式读取:bufio.Scanner

C 处理大文件要么 `fgets` 逐行,要么手写缓冲循环。Go 的 `bufio.Scanner` 是标准答案:内部有缓冲区,一次一行,内存占用与文件大小无关。对照: `os.ReadFile` 读 10GB 日志直接内存爆炸;Scanner 读 10GB 日志内存恒定。

示例 7-2 按行流式读取(内存恒定)

```

package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {
    f, err := os.Open("access.log")
    if err != nil {
        fmt.Println("打开失败:", err)
        return
    }
    defer f.Close()

    scanner := bufio.NewScanner(f)
    lineNum := 0
    for scanner.Scan() { // 每行一次迭代,内存只占一行
        line := scanner.Text()
        lineNum++
        if lineNum <= 3 {
            fmt.Printf("第 %d 行: %s\n", lineNum, line)
        }
    }
    // 循环结束要检查扫描器错误(可能因 IO 故障提前终止)
    if err := scanner.Err(); err != nil {
        fmt.Println("读取中断:", err)
        return
    }
    fmt.Println("总行数:", lineNum)
}

```

⚠ 易错点 7-2:Scanner 默认单行上限 64KB

默认 `bufio.Scanner` 每行最长 64KB,超长行会报 `ErrTooLong` 并停止。处理超长行(如 JSON 日志、大字段)要显式加大: `scanner.Buffer(make([]byte, 1024*1024), 1024*1024)` (初始缓冲 + 最大上限)。生产日志解析器都会设置这个,别等线上爆了才想起。

§7.4 二进制 I/O:encoding/binary

C 直接 `fwrite(&struct)` 的危险:结构体对齐、字节序、`int` 宽度都依赖平台,写出来的文件换台机器就废;还会把填充字节写进去。Go 的做法:字段**逐项显式编码**,指定字节序(大端 `binary.BigEndian` 或小端 `binary.LittleEndian`)。

示例 7-3 二进制读写:固定字节序

```

package main

import (
    "bytes"
    "encoding/binary"
    "fmt"
)

func main() {
    // 写:构造字节流(协议头部 + 数据)
    var buf bytes.Buffer
    ver := uint16(1)
    length := uint32(7)
    _ = binary.Write(&buf, binary.LittleEndian, ver)
    _ = binary.Write(&buf, binary.LittleEndian, length)
    payload := []byte("abcdef")
    _ = binary.Write(&buf, binary.LittleEndian, payload)

    fmt.Printf("编码结果 %d 字节:%x\n", buf.Len(), buf.Bytes())

    // 读:按相同字节序还原
    var v2 uint16
    var l2 uint32
    _ = binary.Read(bytes.NewReader(buf.Bytes()), binary.LittleEndian, &v2)
    _ = binary.Read(bytes.NewReader(buf.Bytes()[2:]), binary.LittleEndian, &l2)
    fmt.Println("还原:", v2, l2)

    // 更精细的写法:手动 Put/Get(性能更好)
    b := make([]byte, 6)
    binary.LittleEndian.PutUint16(b[0:2], 12345)
    binary.LittleEndian.PutUint32(b[2:6], 67890123)
    fmt.Println(binary.LittleEndian.Uint16(b[0:2]), binary.LittleEndian.Uint32(b[2:6]))
}

```

工程建议:二进制协议的三条纪律

①长度字段永远固定宽度(uint16 / uint32),别用 int ;②字节序全局统一,推荐网络序(大端)或与对端约定一致;③每个字段长度与范围要有校验——反序列化是恶意输入的第一道防线,越界长度先拒掉再分配(与第 8 章粘包协议呼应)。

§7.5 路径与目录:filepath 与 os.ReadDir

C 的路径处理是"字符串拼接 + 平台宏": PATH_SEPARATOR 、 strcat ,跨平台要自己折腾。Go 的 path/filepath 一包搞定:拼接、拆分、扩展、清理,自动适配平台分隔符。

示例 7-4 路径与目录操作

```

package main

import (
    "fmt"
    "os"
    "path/filepath"
)

func main() {
    // 拼接:自动处理分隔符(Windows \ 与 Unix / 都正确)
    p := filepath.Join("logs", "2026", "app.log")
    fmt.Println(p)

    // 拆解
    fmt.Println("目录:", filepath.Dir(p))
    fmt.Println("文件名:", filepath.Base(p))
    fmt.Println("扩展名:", filepath.Ext(p))

    // 创建多级目录(已存在也不报错)
    _ = os.MkdirAll(filepath.Join("out", "nested"), 0755)

    // 遍历目录:只读一层
    entries, err := os.ReadDir(".")
    if err != nil {
        fmt.Println("读取目录失败:", err)
        return
    }
    for _, e := range entries {
        kind := "文件"
        if e.IsDir() {
            kind = "目录"
        }
        fmt.Printf("%s(%s)\n", e.Name(), kind)
    }
}

```

⚠ 易错点 7-3:用字符串 "+" 拼路径

`dir + "/" + name` 在 Unix 正常,在 Windows 上 / 通常也能用,但遇到尾斜杠、重复分隔符、`..` 就出幺蛾子。`filepath.Join` 会清理这些;生产代码一律用 `Join`。注意 `filepath.Clean` 还能把 `a../b` 规范化。

§7.6 JSON 序列化:encoding/json

C 的标准库没有 JSON;Go 的 `encoding/json` 是官方标准实现,配合第 1 章的 `struct` 与第 12 章的 `struct tag` 使用。三个核心 API: `Marshal` (结构体 → 字节)、`Unmarshal` (字节 → 结构体)、`MarshalIndent` (格式化输出);流式用 `Encoder / Decoder` (第 8 章 HTTP 里会反复出现)。

示例 7-5 JSON 序列化与反序列化

```

package main

import (
    "encoding/json"
    "fmt"
)

// 字段名由 tag 控制;未导出字段不参与序列化
type User struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
    Tags []string `json:"tags,omitempty"` // 空值省略
    Score float64 `json:"score"`
}

func main() {
    u := User{Name: "张三", Age: 28, Score: 92.5}

    // 序列化:紧凑输出
    data, err := json.Marshal(u)
    if err != nil {
        fmt.Println("序列化失败:", err)
        return
    }
    fmt.Println(string(data)) // {"name":"张三","age":28,"score":92.5}(tags 省略)

    // 格式化输出(配置/日志友好)
    pretty, _ := json.MarshalIndent(u, "", " ")
    fmt.Println(string(pretty))

    // 反序列化
    raw := []byte(`{"name":"李四","age":30,"score":88.0,"extra":"忽略我"}`)
    var v User
    if err := json.Unmarshal(raw, &v); err != nil {
        fmt.Println("反序列化失败:", err)
        return
    }
    fmt.Printf("还原: %v\n", v) // extra 字段被忽略

    // 任意结构:map 与接口
    var m map[string]any
    _ = json.Unmarshal(raw, &m)
    fmt.Println(m["name"], m["age"])
}

```

⚠ 易错点 7-4:反序列化忽略未知字段与类型不匹配

`Unmarshal` 默认**忽略** JSON 中的未知字段,字段类型不匹配(如字符串给了数字)会返回错误。生产上:①需要严格校验时用 `Decoder.DisallowUnknownFields()`;②数字一律用 `float64` 接收再转换(JSON 没有整数概念, `json.Number` 可保精度);③日期不是内建类型,用 `string` + 自定义 `UnmarshalJSON`(第 12 章展开)。

§7.7 标准流与命令行参数

对照 C: `stdin/stdout/stderr` → Go 的 `os.Stdin / os.Stdout / os.Stderr` (类型 `*os.File`, 也是 `io.Reader / io.Writer`); `argc/argv` → `os.Args` (切片,第 0 项是程序名);手写 `getopt` → `flag` 标准包。

示例 7-6 `os.Args` 与 `flag` 包

```
package main

import (
    "flag"
    "fmt"
    "os"
)

func main() {
    // os.Args:全部参数(第 0 项是程序名)
    fmt.Println("程序名:", os.Args[0])
    fmt.Println("参数个数:", len(os.Args)-1)

    // flag 包:命名参数解析
    name := flag.String("name", "匿名", "用户名")
    verbose := flag.Bool("v", false, "详细输出")
    port := flag.Int("port", 8080, "端口")
    flag.Parse() // 必须调用,才会解析

    fmt.Printf("name=%s verbose=%v port=%d\n", *name, *verbose, *port)

    // 位置参数:flag.Args() 返回剩余参数
    rest := flag.Args()
    fmt.Println("剩余位置参数:", rest)
}
```

示例 7-7 标准流:行过滤工具(类似 `grep` 的骨架)


```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

func main() {
    // 从 stdin 逐行读,过滤含关键字(第 0 个参数)的行,写回 stdout
    keyword := "ERROR"
    if len(os.Args) > 1 {
        keyword = os.Args[1]
    }

    scanner := bufio.NewScanner(os.Stdin)
    for scanner.Scan() {
        line := scanner.Text()
        if strings.Contains(line, keyword) {
            fmt.Println(line) // 输出到 stdout
        }
    }
    if err := scanner.Err(); err != nil {
        fmt.Fprintln(os.Stderr, "读取 stdin 失败:", err) // 错误走 stderr
        os.Exit(1)
    }
}
```

§7.8 CSV 与日志:生产场景

CSV 别手写 split: `encoding/csv` 处理引号、逗号内换行等边界。日志文件用追加模式 + 可选轮转(生产上交给 `lumberjack` 等库,原理见 §7.2 的 `OpenFile` 追加)。

示例 7-8 CSV 读写

```

package main

import (
    "encoding/csv"
    "fmt"
    "os"
    "strconv"
)

func main() {
    // 写 CSV
    f, err := os.Create("scores.csv")
    if err != nil {
        fmt.Println("创建失败:", err)
        return
    }
    defer f.Close()
    w := csv.NewWriter(f)
    _ = w.Write([]string{"姓名", "分数"})
    _ = w.Write([]string{"张三", "92"})
    _ = w.Write([]string{"李四", "88"})
    w.Flush()
    if err := w.Error(); err != nil {
        fmt.Println("写 CSV 失败:", err)
    }

    // 读 CSV 并求和
    rf, err := os.Open("scores.csv")
    if err != nil {
        fmt.Println("打开失败:", err)
        return
    }
    defer rf.Close()
    r := csv.NewReader(rf)
    records, _ := r.ReadAll()
    total := 0
    for _, rec := range records[1:] { // 跳过表头
        n, _ := strconv.Atoi(rec[1])
        total += n
    }
    fmt.Println("总分:", total)
}

```

本章速查表

主题	要点
读小文件	<code>os.ReadFile(path) → ([]byte, error)</code> ;自动打开+关闭
写小文件	<code>os.WriteFile(path, data, 0644)</code> ;覆盖写,不存在则创建
追加	<code>os.OpenFile(path, os.O_APPEND os.O_CREATE os.O_WRONLY, 0644)</code>
流式读行	<code>bufio.Scanner</code> ;记得 <code>scanner.Err()</code> ;超长行调 Buffer
底层读写	<code>f.Read(buf) → (n, err)</code> ;EOF 是 <code>io.EOF</code> ;要循环读满
关闭	打开后立刻 <code>defer f.Close()</code>
错误分类	<code>errors.Is(err, os.ErrNotExist)</code> ;勿比对错误字符串
二进制	<code>encoding/binary</code> :LittleEndian/BigEndian;显式逐字段编码
路径	<code>filepath.Join/Dir/Base/Ext/Clean</code> ;跨平台分隔符
目录	<code>os.ReadDir(dir) → 条目列表</code> ; <code>os.MkdirAll</code> 多级创建
JSON	<code>json.Marshal/Unmarshal/MarshalIndent</code> ;tag 控字段名;Decoder 流式
标准流	<code>os.Stdin/Stdout/Stderr</code> ;错误写 <code>stderr</code> 并 <code>os.Exit(1)</code>
命令行	<code>os.Args</code> 全参数; <code>flag</code> 包解析命名参数
CSV	<code>encoding/csv</code> 读写;别手写 <code>split</code> 处理引号

最小必会清单

- 能写出"读文件 + 判错 + 打印"的三行式,以及"写文件"的三行式
- 能解释 Scanner 流式读大文件为什么内存恒定,以及 64KB 限制与调大方法
- 能写出追加模式 OpenFile 的参数组合,并用 defer 关闭
- 能解释"为什么不能 fwrite 结构体",写出 binary 编码/解码
- 能写出 filepath.Join 与 os.ReadDir 的完整目录遍历
- 能写出带 tag 的结构体与 Marshal/Unmarshal 往返
- 能说出 flag 包的三步用法(定义/Parse/取值)
- 能写出"stdin 过滤 → stdout 输出、错误走 stderr"的小工具骨架

自测题

Q 1. 读一个 100MB 的文本文件,要求内存占用与文件大小无关,写出代码骨架(打开、扫描、关闭、错误检查)。

✅ 参考答案

```
f, err := os.Open(fn); if err != nil { ... }; defer f.Close(); scanner := bufio.NewScanner(f); for scanner.Scan() { 处理 scanner.Text() }; if err := scanner.Err(); err != nil { ... }。Scanner 内部固定缓冲,逐行处理,内存与文件大小无关。
```

Q 2. `os.WriteFile("a.txt", data, 0644)` 与 `os.OpenFile("a.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)` 的行为区别是什么?

✓ 参考答案

前者是"不存在创建、存在截断"(等价 `O_TRUNC`);后者是追加写(`O_APPEND`),不截断,保留原内容。追加日志用后者,覆盖配置文件用前者。

Q 3. JSON 反序列化时,输入 `{"name":"张三","age":"28"}` 到 `struct{ Age int }` 会怎样?

✓ 参考答案

返回错误:字符串不能反序列化为 `int` 字段(`json: cannot unmarshal string into Go struct field ... of type int`),且 `v` 保持调用前的值(`Unmarshal` 失败不清空)。生产上要注意"解析失败时的部分赋值"——先解码到新变量,成功后再使用。

Q 4. 为什么 C 的 `fwrite(&struct)` 有跨平台问题,Go 的 `binary` 为什么没有?

✓ 参考答案

C 直接写结构体把"内存布局"写进文件:结构体对齐规则、字段顺序、`int` 宽度、字节序全由平台决定,换编译器/换 CPU 就废;还可能把填充字节写进去。Go 的 `encoding/binary` 逐字段、显式指定宽度与字节序,文件格式由代码决定而非平台决定。

Q 5. 写出 `flag` 包解析 `-name 张三 -v` 的完整流程,并说明 `flag.Args()` 的用途。

✓ 参考答案

`name := flag.String("name", "匿名", "说明"); verbose := flag.Bool("v", false, "详细"); flag.Parse()`, 取值用 `*name`、`*verbose`。`flag.Args()` 返回 `Parse` 后剩余的位置参数(非 `-` 开头的参数),如文件名列表。

Q 6. `Scanner` 遇到超长行会怎样?如何解决?

✓ 参考答案

默认每行上限 64KB,超长触发 `ErrTooLong`,`Scan` 返回 `false` 停止。解决: `scanner.Buffer(make([]byte, 1<<20), 1<<20)` 调大(首参初始缓冲,次参上限)。另外, `ScanLines` 只认 `\n`;处理 `\r\n` 时 `Scanner` 已自动去 `\r`。

章末实验题:日志解析与统计工具

实验 7:日志解析统计工具

实验目标:实现一个日志分析工具:读取"时间|路径|状态码|耗时ms"格式的访问日志,统计总量、平均耗时、状态码分布、TOP 路径,输出格式化 JSON 报告到 out 目录。

实验要求:

- 任务 1:用 `os.Open` + `bufio.Scanner` 流式读取日志文件(覆盖:\$7.1、\$7.3 流式读取)
- 任务 2:每行按 `|` 分隔解析为结构体,坏行跳过并计数(覆盖:\$7.2 文本处理、字符串与转换)
- 任务 3:统计状态码分布(map 聚合)、各路径访问次数(覆盖:\$7.8 统计、map)
- 任务 4:平均耗时与错误率计算,注意显式类型转换(覆盖:\$7.2、第 1 章转换)
- 任务 5:TOP 5 路径排序输出(覆盖:sort 切片)
- 任务 6:用 `os.MkdirAll` + `filepath.Join` 建 out 目录并写报告(覆盖:\$7.5 路径目录)
- 任务 7:报告用 `json.MarshalIndent` 格式化输出(覆盖:\$7.6 JSON)
- 任务 8:支持命令行参数指定日志文件,错误走 `stderr` 并返回非零退出码(覆盖:\$7.7 参数与标准流)

实验环境:Go 1.21+, `go run main.go logs.txt` ;自行生成测试日志验证。

第一步 写 `parseLine` 函数,单测它的四种输入:正常行、缺字段行、非数字行、空行

第二步 流程:打开 → `Scanner` 循环 → 聚合 map → 排序 → 写报告

第三步 告结构体定义好 tag,方便直接 `Marshal`

第四步 后加 `os.Args` 参数与 `stderr` 错误处理

✔ 参考答案要点

```
// 日志解析统计工具:流式读取 + 聚合统计 + JSON 报告
package main

import (
    "bufio"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "sort"
    "strconv"
    "strings"
)

// 一行日志:时间|路径|状态码|耗时ms
type logLine struct {
    Time string
    Path string
    Code int
    Cost int
}

// 统计报告(带 JSON tag,$7.6)
type report struct {
    Total      int           `json:"total"`
    AvgCost    float64       `json:"avg_cost"`
    ErrorRate  float64       `json:"error_rate"`
    ByCode     map[string]int `json:"by_code"`
    TopPaths   []pathCount   `json:"top_paths"`
}

type pathCount struct {
    Path  string `json:"path"`
    Count int    `json:"count"`
}

// parseLine 解析一行;格式错误返回 false(坏行计数由调用方负责)
func parseLine(line string) (logLine, bool) {
    parts := strings.Split(line, "|")
    if len(parts) != 4 {
        return logLine{}, false
    }
    code, err1 := strconv.Atoi(parts[2])
    cost, err2 := strconv.Atoi(parts[3])
    if err1 != nil || err2 != nil {
        return logLine{}, false
    }
    return logLine{Time: parts[0], Path: parts[1], Code: code, Cost: cost}, true
}

// analyze 流式统计主逻辑
func analyze(src string) (*report, error) {
    f, err := os.Open(src)
    if err != nil {
        return nil, fmt.Errorf("打开日志失败: %w", err)
    }

```

```

}
defer f.Close() // 无论成败,句柄必释放

var r report
r.ByCode = map[string]int{}
pathCounts := map[string]int{}
totalCost := 0
errors := 0
badLines := 0

scanner := bufio.NewScanner(f)
scanner.Buffer(make([]byte, 64*1024), 1024*1024) // 允许超长行
for scanner.Scan() {
    ll, ok := parseLine(scanner.Text())
    if !ok {
        badLines++
        continue // 坏行跳过,不影响整体
    }
    r.Total++
    totalCost += ll.Cost
    pathCounts[ll.Path]++
    code := strconv.Itoa(ll.Code)
    r.ByCode[code]++
    if ll.Code >= 500 {
        errors++
    }
}
if err := scanner.Err(); err != nil {
    return nil, fmt.Errorf("读取中断: %w", err)
}

if r.Total > 0 {
    r.AvgCost = float64(totalCost) / float64(r.Total) // 显式转换
    r.ErrorRate = float64(errors) / float64(r.Total) * 100
}
for p, c := range pathCounts {
    r.TopPaths = append(r.TopPaths, pathCount{Path: p, Count: c})
}
sort.Slice(r.TopPaths, func(i, j int) bool {
    return r.TopPaths[i].Count > r.TopPaths[j].Count
})
if len(r.TopPaths) > 5 {
    r.TopPaths = r.TopPaths[:5] // 只保留 TOP 5
}
fmt.Printf("坏行 %d 条(已跳过)\n", badLines)
return &r, nil
}

func main() {
    // 命令行参数:默认 logs.txt($7.7)
    src := "logs.txt"
    if len(os.Args) > 1 {
        src = os.Args[1]
    }
}

```



```

r, err := analyze(src)
if err != nil {
    fmt.Fprintln(os.Stderr, "分析失败:", err) // 错误走 stderr
    os.Exit(1)                                // 非零退出码
}

// JSON 报告:格式化输出($7.6)
data, err := json.MarshalIndent(r, "", " ")
if err != nil {
    fmt.Fprintln(os.Stderr, "JSON 编码失败:", err)
    os.Exit(1)
}

// 输出到 out 目录($7.5)
outDir := "out"
if err := os.MkdirAll(outDir, 0755); err != nil {
    fmt.Fprintln(os.Stderr, "创建目录失败:", err)
    os.Exit(1)
}
outPath := filepath.Join(outDir, "report.json")
if err := os.WriteFile(outPath, data, 0644); err != nil {
    fmt.Fprintln(os.Stderr, "写报告失败:", err)
    os.Exit(1)
}
fmt.Println("报告已生成:", outPath)
fmt.Println(string(data))
}

```

设计说明:实验把本章 8 节全部织入:Scanner 流式读与 Buffer 调大(\$7.3)、OpenFile 打开与 defer 关闭(\$7.2)、按行解析与坏行容错(\$7.2)、map 聚合与排序(\$7.8)、JSON tag 与 MarshalIndent(\$7.6)、MkdirAll 与 filepath.Join(\$7.5)、os.Args 与 stderr/exit(\$7.7)。测试数据可现场生成: `for i in {1..100}; do echo "2026-08-26 10:00:00$i|/api/user|200|12"; done > logs.txt`。扩展:为 analyze 写单元测试(用 t.TempDir 生成临时日志),再加一个 `-out` flag 指定输出目录(覆盖 flag 包)。

下一章预告:第8章 网络通信。文件是"本地通道",下一章走出机器:从 C 的 socket/htons 到 Go 的 net 包,goroutine per connection 模式、UDP、HTTP 客户端与服务端,以及粘包/拆包协议设计——每个示例都能直接抄进生产。